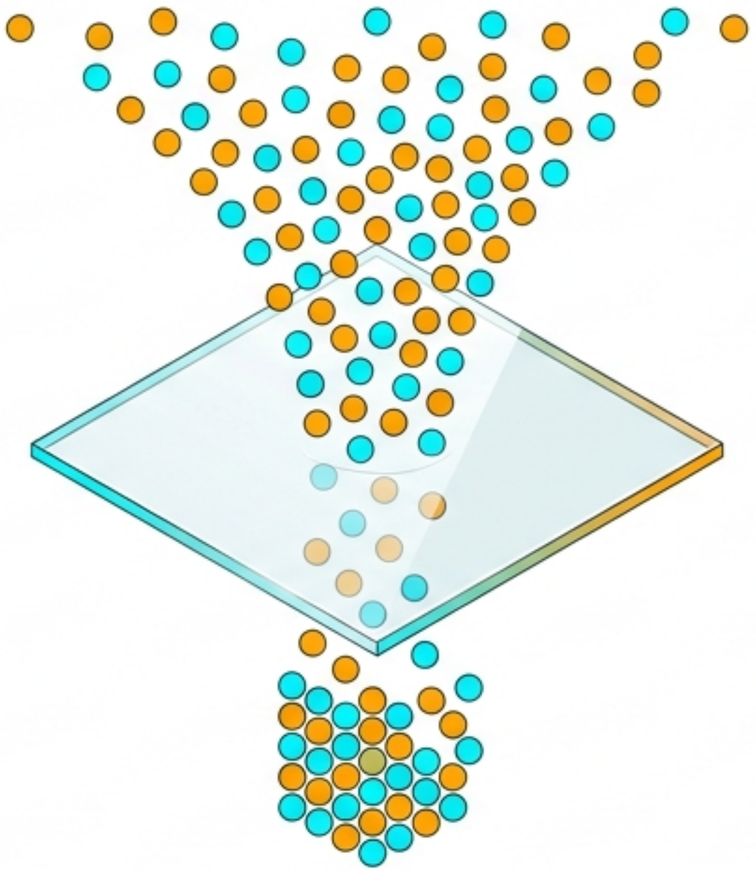


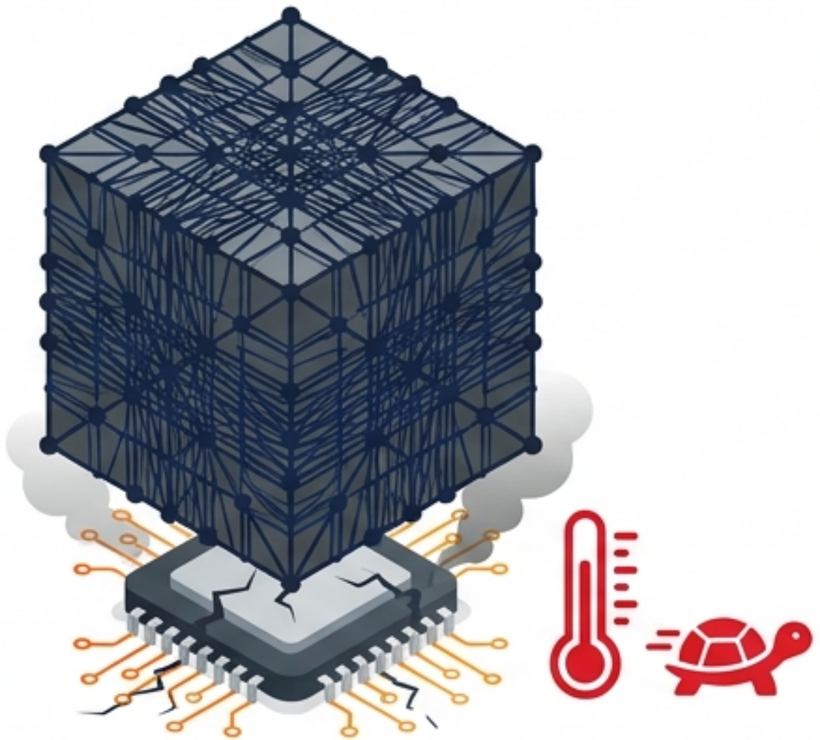
PHÉP CHIẾU NGẪU NHIÊN

Lối tắt toán học thông minh để xử lý dữ liệu siêu lớn mà không cần huấn luyện.

Gemini Notebook

Tại sao PCA truyền thống lại "hụt hơi"?

VẤN ĐỀ CỦA PCA TRUYỀN THỐNG



Với **dữ liệu siêu cao chiều** (hàng triệu đặc trưng), PCA bắt buộc phải **tính toán SVD** cực kỳ phức tạp. Kết quả: **Quá tải tài nguyên** & **Tốc độ chậm chạp**.

GIẢI PHÁP ĐỘT PHÁ

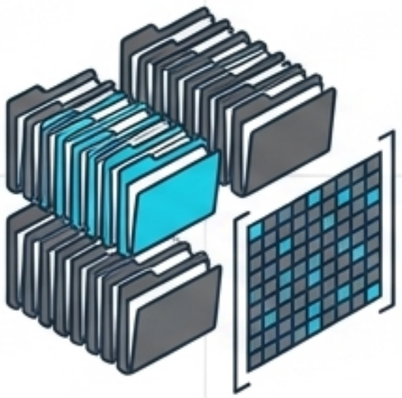


Chúng ta cần một **phương pháp đột phá**: **Giữ nguyên cấu trúc dữ liệu** nhưng **KHÔNG CẦN tìm trục tối ưu**.

Gemini Notebook

Cơ chế "Chiếu mà không cần học"

Thuật toán **KHÔNG HỀ** nhìn vào dữ liệu để học.
Chỉ cần sinh một ma trận ngẫu nhiên **P** và nhân trực tiếp.



Ma trận dữ liệu **X**





Ma trận ngẫu nhiên **P**
(d × n)



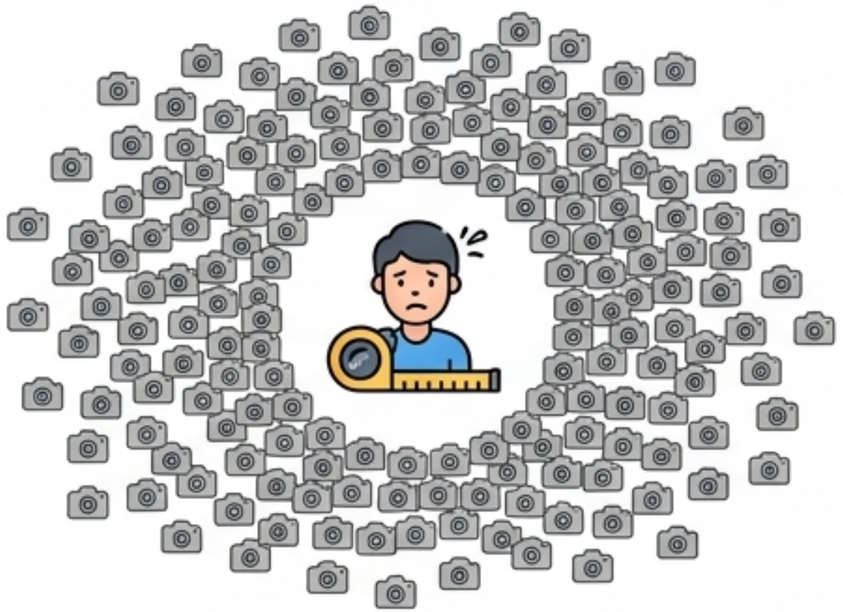


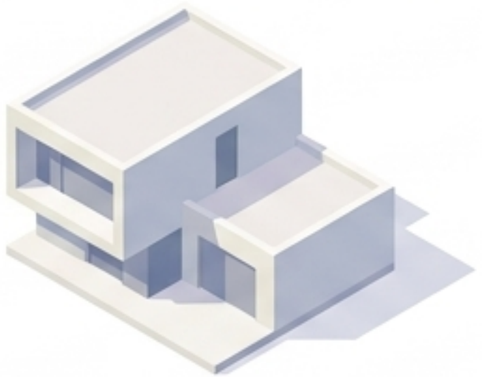
Dữ liệu thu gọn
X_reduced

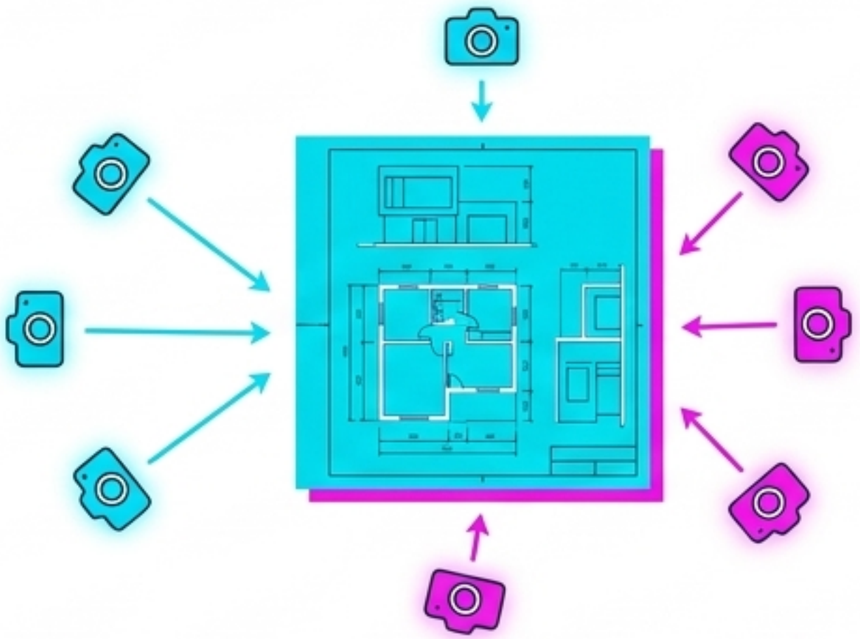
$$X_reduced = X \cdot P^T$$

Gemini Notebook

Hiểu đơn giản qua "Góc chụp ngôi nhà"







Cố gắng đo đạc và đánh giá **20.000 góc** để tìm ra góc đẹp nhất -> **Rất chậm.**

"Nhắm mắt bấm máy" ngẫu nhiên **7.000** góc độc lập -> **Tốc độ chớp nhoáng!**

Khi số lượng góc ngẫu nhiên đủ lớn, khoảng cách và tỷ lệ chi tiết của ngôi nhà vẫn được giữ nguyên vẹn một cách kỳ diệu.

Gemini Notebook

Bí mật toán học: Định lý Johnson-Lindenstrauss (JL)

Khi chiếu dữ liệu từ không gian chiều cao xuống không gian d chiều bằng phép chiếu ngẫu nhiên...



Khoảng cách bình phương giữa 2 điểm bất kỳ hầu như không bị biến dạng

(trong một dung sai ε cho phép, với xác suất rất cao)

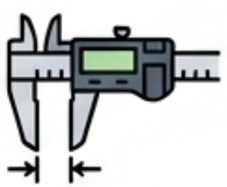
Gemini Notebook

Số chiều mục tiêu (d) được quyết định như thế nào?



Số lượng **mẫu dữ liệu**

$$d \geq \frac{4\log(m)}{\frac{1}{2}\epsilon^2 - \frac{1}{3}\epsilon^3}$$



Dung sai **biến dạng khoảng cách**

Điều kỳ diệu nhất? Công thức này **HOÀN TOÀN KHÔNG chứa biến n**! Số chiều gốc không hề ảnh hưởng đến **d**.



Gemini Notebook

Sự kỳ diệu của Định lý JL qua những con số

Bài toán: Cho **m = 5.000** mẫu, dung sai **ε = 10%** (0.1).

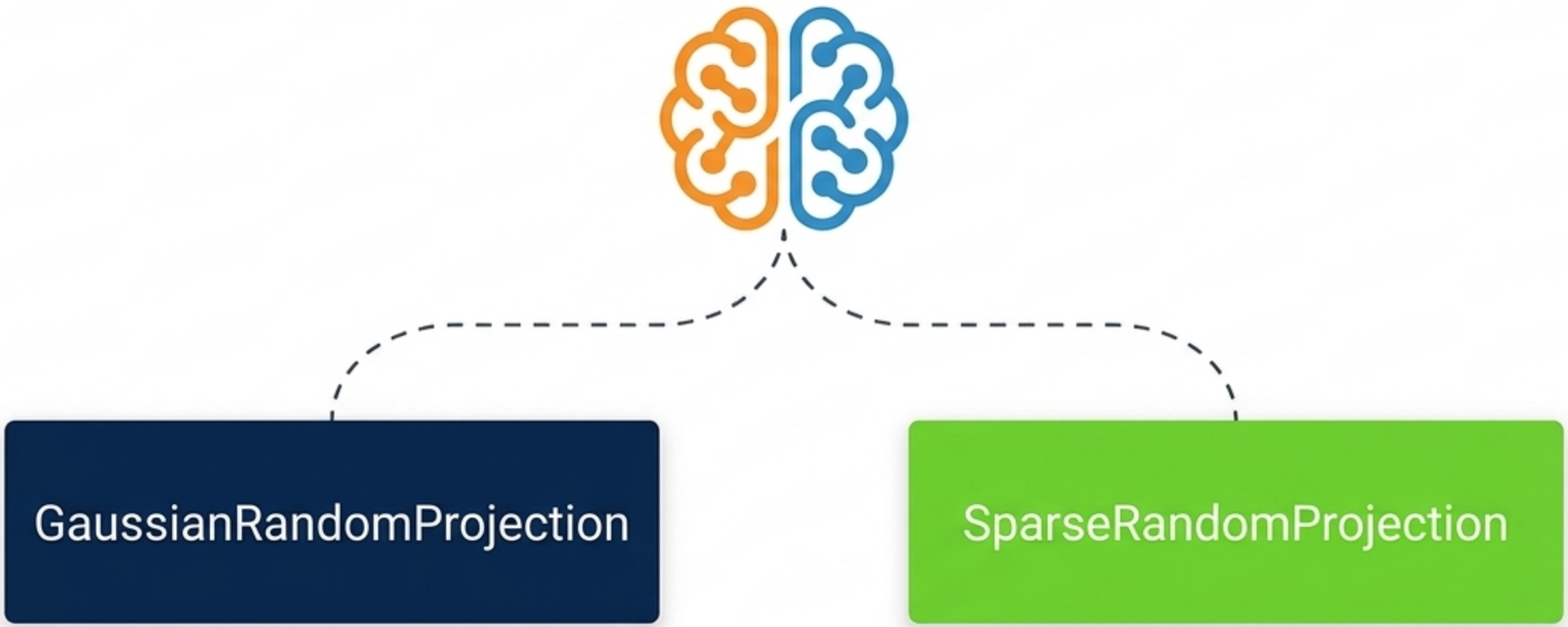


Dù dữ liệu gốc của bạn có 20 nghìn hay 1 triệu chiều, bạn vẫn chỉ cần đúng 7.300 chiều để bảo toàn hoàn toàn thông tin.

Gemini Notebook

Thực hành bộ biến đổi (Transformers) trong Scikit-Learn

Scikit-Learn cung cấp 2 giải pháp chính để thực thi phép chiếu ngẫu nhiên:

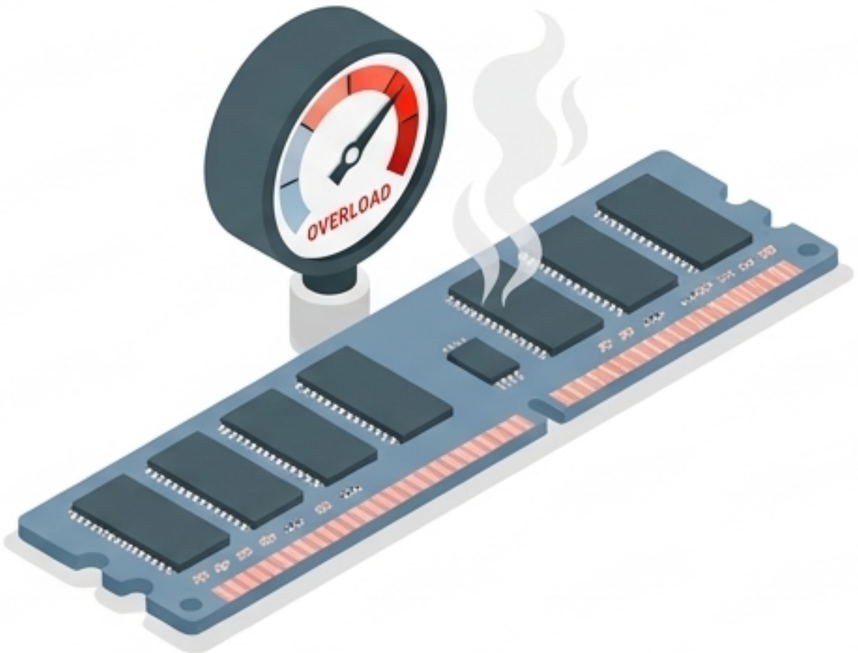


Gemini Notebook

Giải pháp 1: Gaussian Random Projection



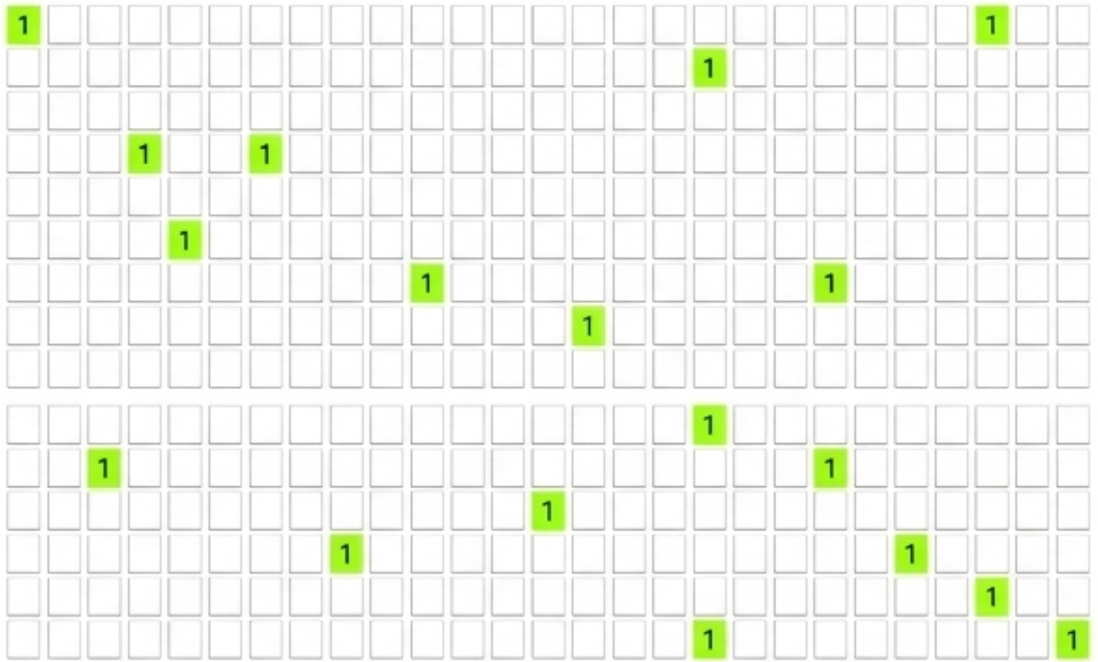
Ma trận đặc (Dense Matrix) - Mọi ô đều chứa dữ liệu.



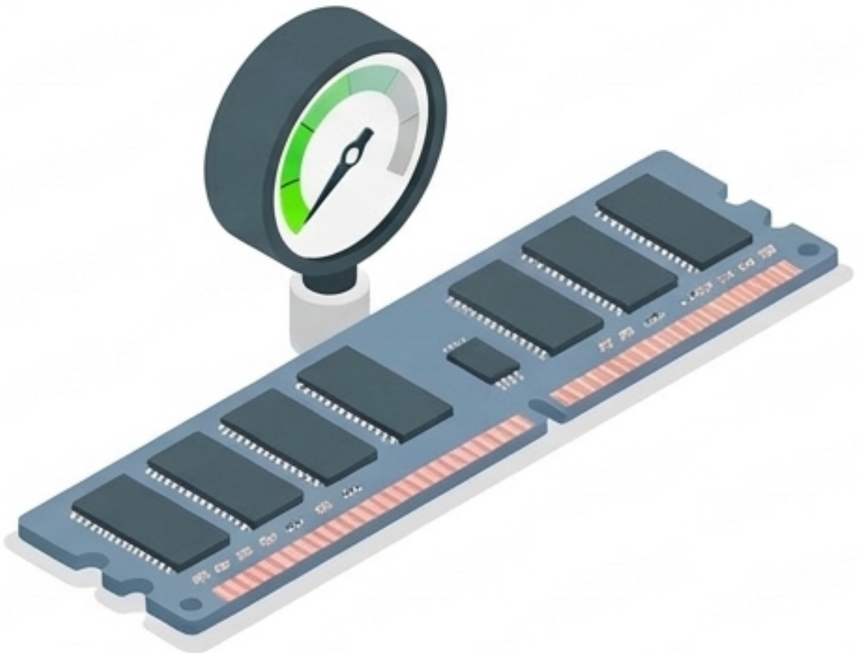
- **Cơ chế:** Sinh ma trận **P** lấy mẫu độc lập từ phân phối chuẩn Gaussian N(0, 1/d).
- **Điểm yếu cốt tử:** Tạo ra Ma trận đặc (Dense) - Mọi ô đều chứa dữ liệu.
- **Ví dụ thực tế:** Với m=5000, n=20.000, d=7.300 -> Thuật toán này sẽ nuốt trọn ~1.2 GB RAM chỉ để lưu trữ ma trận chiếu!

Gemini Notebook

Giải pháp tối ưu: Sparse Random Projection



Ma trận đặc (Sparse Matrix) – Mọi ô đều chứa dữ liệu.



- **Cơ chế:** Sinh Ma trận thưa (Sparse matrix), ép đại đa số các ô về giá trị 0.
- **Công thức mật độ:** $r = 1/\sqrt{n}$.
- **Ví dụ trực quan:** Với n = 20.000, trung bình chỉ có 1 trong 141 ô có giá trị khác 0.

Gemini Notebook

Bảng Đối Đầu: Gaussian vs. Sparse

	GaussianRandomProjection	SparseRandomProjection
Loại ma trận	Ma trận đặc (Dense)	**Ma trận thưa** (Sparse)
Dung lượng RAM	Rất lớn (~1.2 GB)	**Rất nhỏ** (~25 MB)
Tốc độ tính toán	Chậm hơn	**Nhanh hơn** ~50%
Bảo toàn khoảng cách	Rất tốt	**Tương đương** Gaussian
Xử lý Input thưa	Ép thành ma trận đặc ❌	**Bảo toàn định dạng thưa** ✅

Gemini Notebook

Khuyến nghị thực chiến (The Golden Rule)



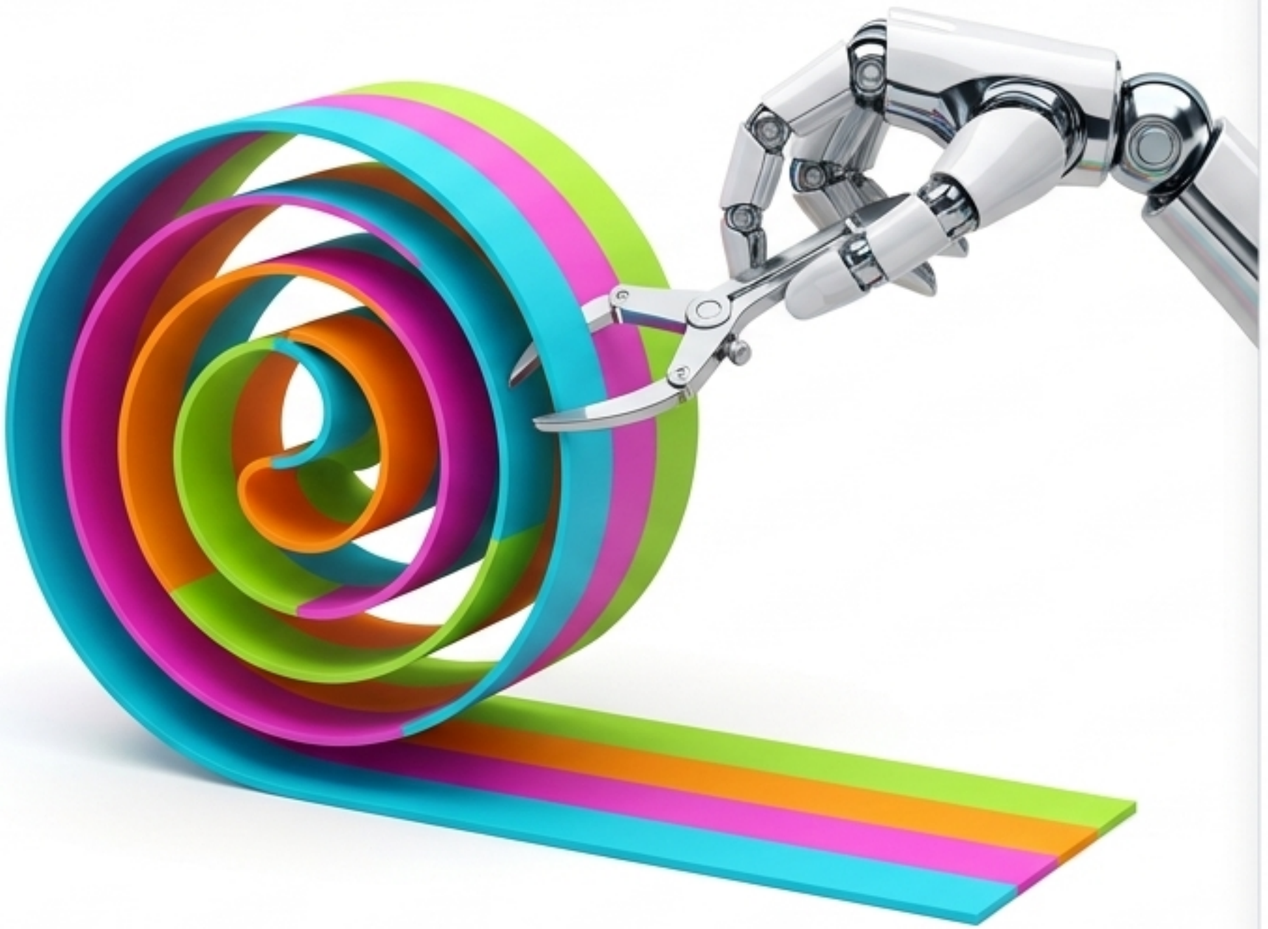
SparseRandomProjection

Nguyên tắc: Trong hầu hết mọi bài toán thực tế (đặc biệt với Dữ liệu lớn hoặc Dữ liệu thưa), hãy luôn đặt SparseRandomProjection làm ưu tiên số 1.

- Tối ưu sức mạnh CPU (**nhanh hơn 50%**)
- Giải phóng hoàn toàn gánh nặng bộ nhớ RAM (**25MB vs 1.2GB**)
- Đảm bảo **độ chính xác toán học xuất sắc**

Gemini Notebook

Chờ đã, nếu **dữ liệu bị "xoắn"** thì sao?



Phép chiếu ngẫu nhiên rất xuất sắc với **dữ liệu tuyến tính**. Nhưng nếu **cấu trúc dữ liệu bị cuộn xoắn phi tuyến tính** (như một cuộn bánh Swiss Roll), chiếu ngẫu nhiên sẽ **phá hỏng hoàn toàn cấu trúc!**

Cùng khám phá phần tiếp theo:
Phần 4: Nhúng tuyến tính cục bộ (LLE) để học cách nghệ thuật "mở cuộn" dữ liệu.

Gemini Notebook